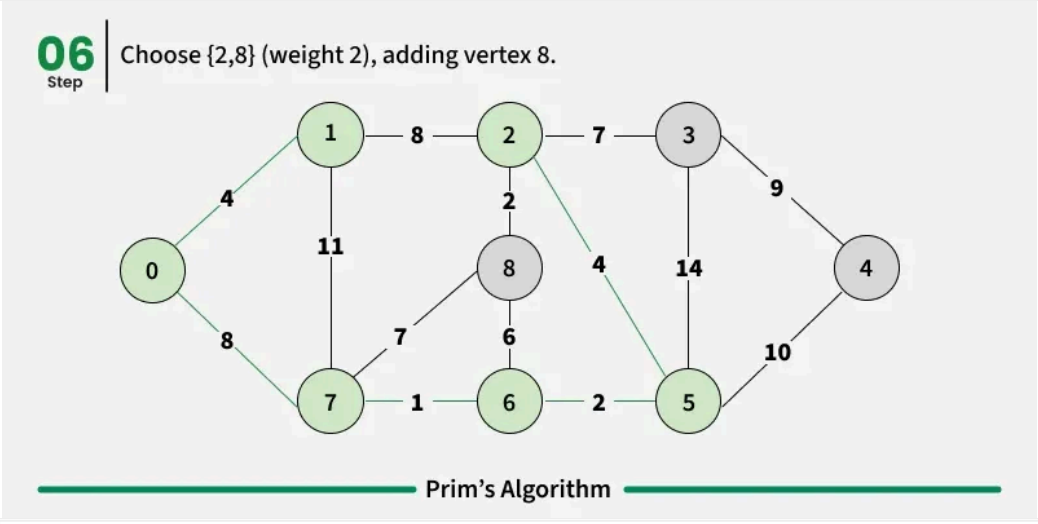
Prim’s Algorithm for Minimum Spanning Tree (MST)

Last Updated : 20 Dec, 2025

Prim’s algorithm is a Greedy algorithm like Kruskal's algorithm. This algorithm always starts with a single node and moves through several adjacent nodes, in order to explore all of the connected edges along the way.

- The algorithm starts with an empty spanning tree.
- The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, and the other set contains the vertices not yet included.
- At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.



Working of the Prim's Algorithm

rbitrary vertex as the starting vertex of the MST. We pick 0 in the above diagram.
2. Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).
3. Find edges connecting any tree vertex with the fringe vertices.
4. Find the minimum among these edges.
5. Add the chosen edge to the MST. Since we consider only the edges that connect fringe vertices with the rest, we never get a cycle.
6. Return the MST and exit

How to implement Prim's Algorithm?

Follow the given steps to utilize the **Prim's Algorithm** mentioned above for finding MST of a graph:

- Create a set **mstSet** that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign the key value as 0 for the first vertex so that it is picked first.
- While **mstSet** doesn't include all vertices
    - Pick a vertex **u** that is not there in **mstSet** and has a minimum key value.
    - Include **u** in the **mstSet**.
    - Update the key value of all adjacent vertices of **u**. To update the key values, iterate through all adjacent vertices. For every adjacent vertex **v**, if the weight of edge **u-v** is less than the previous key value of **v**, update the key value as the weight of **u-v**.

The idea of using key values is to pick the minimum weight edge from the cut. The key values are used only for vertices that are not yet included in MST, the key value for these vertices indicates the minimum weight edges connecting them to the set of vertices included in MST.

C++ C Java Python C# JavaScript

```cpp
// A C++ program for Prim's Minimum
// Spanning Tree (MST) algorithm. The program is
// for adjacency matrix representation of the graph
#include <bits/stdc++.h>
using namespace std;

// A utility function to find the vertex with
// minimum key value, from the set of vertices
// not yet included in MST
int minKey(vector<int> &key, vector<bool> &mstSet) {

    // Initialize min value
    int min = INT_MAX, min_index;

    for (int v = 0; v < mstSet.size(); v++)
        if (mstSet[v] == false && key[v] < min)
            min = key[v], min_index = v;

    return min_index;
}

// A utility function to print the
// constructed MST stored in parent[]
void printMST(vector<int> &parent, vector<vector<int>> &gra
{
    cout << "Edge \tWeight\n";
    for (int i = 1; i < graph.size(); i++)
        cout << parent[i] << " - " << i << " \t"
             << graph[parent[i]][i] << " \n";
}

// Function to construct and print MST for
// a graph represented using adjacency
// matrix representation
void primMST(vector<vector<int>> &graph) {

    int V = graph.size();

    // Array to store constructed MST
    vector<int> parent(V);
```

```
    ues used to pick minimum weight edge in cut
    vector<int> key(V);

    // To represent set of vertices included in MST
    vector<bool> mstSet(V);

    // Initialize all keys as INFINITE
    for (int i = 0; i < V; i++)
        key[i] = INT_MAX, mstSet[i] = false;

    // Always include first 1st vertex in MST.
    // Make key 0 so that this vertex is picked as first
    // vertex.
    key[0] = 0;

    // First node is always root of MST
    parent[0] = -1;

    // The MST will have V vertices
    for (int count = 0; count < V - 1; count++) {

        // Pick the minimum key vertex from the
        // set of vertices not yet included in MST
        int u = minKey(key, mstSet);

        // Add the picked vertex to the MST Set
        mstSet[u] = true;

        // Update key value and parent index of
        // the adjacent vertices of the picked vertex.
        // Consider only those vertices which are not
        // yet included in MST
        for (int v = 0; v < V; v++)

            // graph[u][v] is non zero only for adjacent
            // vertices of m mstSet[v] is false for vertice
            // not yet included in MST Update the key only
            // if graph[u][v] is smaller than key[v]
            if (graph[u][v] && mstSet[v] == false
                && graph[u][v] < key[v])
                parent[v] = u, key[v] = graph[u][v];
    }
```

```
he constructed MST
    printMST(parent, graph);
}

// Driver's code
int main() {
    vector<vector<int>> graph = { { 0, 2, 0, 6, 0 },
                                  { 2, 0, 3, 8, 5 },
                                  { 0, 3, 0, 0, 7 },
                                  { 6, 8, 0, 0, 9 },
                                  { 0, 5, 7, 9, 0 } };

    // Print the solution
    primMST(graph);

    return 0;
}
```

Output

```
Edge    Weight
0 - 1   2
1 - 2   3
0 - 3   6
1 - 4   5
```

Time Complexity: $O(V^2)$, As, we are using adjacency matrix, if the input graph is represented using an adjacency list, then the time complexity of Prim's algorithm can be reduced to O((E+V) * logV) with the help of a binary heap.
Auxiliary Space: O(V)

Optimized Implementation using Adjacency List Representation (of Graph) and Priority Queue

1. We transform the adjacency matrix into adjacency list using **ArrayList<ArrayList<Integer>>**. in Java, list of list in Python and array of vectors in C++.
2. Then we create a Pair class to store the vertex and its weight .

on the basis of lowest weight.

4. We create priority queue and push the first vertex and its weight in the queue
5. Then we just traverse through its edges and store the least weight in a variable called **ans.**
6. At last after all the vertex we return the ans.

C++ Java Python C# JavaScript

```cpp
#include<bits/stdc++.h>
using namespace std;

// Function to find sum of weights of edges of the Minimum
int spanningTree(int V, int E, vector<vector<int>> &edges)

    // Create an adjacency list representation of the graph
    vector<vector<int>> adj[V];

    // Fill the adjacency list with edges and their weights
    for (int i = 0; i < E; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int wt = edges[i][2];
        adj[u].push_back({v, wt});
        adj[v].push_back({u, wt});
    }

    // Create a priority queue to store edges with their we
    priority_queue<pair<int,int>, vector<pair<int,int>>, gr

    // Create a visited array to keep track of visited vert
    vector<bool> visited(V, false);

    // Variable to store the result (sum of edge weights)
    int res = 0;

    // Start with vertex 0
    pq.push({0, 0});

    // Perform Prim's algorithm to find the Minimum Spannir
    while(!pq.empty()){
        auto p = pq.top();
```

```
();

        int wt = p.first;  // Weight of the edge
        int u = p.second;  // Vertex connected to the edge

        if(visited[u] == true){
            continue;  // Skip if the vertex is already vis
        }

        res += wt;  // Add the edge weight to the result
        visited[u] = true;  // Mark the vertex as visited

        // Explore the adjacent vertices
        for(auto v : adj[u]){
            // v[0] represents the vertex and v[1] represer
            if(visited[v[0]] == false){
                pq.push({v[1], v[0]});  // Add the adjacent
queue
            }
        }
    }

    return res;  // Return the sum of edge weights of the M
}

int main() {
    vector<vector<int>> graph = {{0, 1, 5},
                                 {1, 2, 3},
                                 {0, 2, 1}};

    cout << spanningTree(3, 3, graph) << endl;

    return 0;
}
```

Output

4

Time Complexity: O((E+V)*log(V)) where V is the number of vertex and E is the number of edges

O(E+V) where V is the number of vertex and E is the number of edges

Advantages and Disadvantages of Prim's algorithm

Advantages:

- Prim's algorithm is guaranteed to find the MST in a connected, weighted graph.
- It has a time complexity of O((E+V)*log(V)) using a binary heap or Fibonacci heap, where E is the number of edges and V is the number of vertices.
- It is a relatively simple algorithm to understand and implement compared to some other MST algorithms.

Disadvantages:

- Like Kruskal's algorithm, Prim's algorithm can be slow on dense graphs with many edges, as it requires iterating over all edges at least once.
- Prim's algorithm relies on a priority queue, which can take up extra memory and slow down the algorithm on very large graphs.
- The choice of starting node can affect the MST output, which may not be desirable in some applications.

Also Check:

- *Kruskal's Algorithm for MST*
- *Minimum cost to connect all cities*
- *Minimum cost to provide water*
- *Second Best Minimum Spanning Tree*
- *Check if an edge is a part of any MST*
- *Minimize count of connections*

MP4 File 65.10 MB

VTT Subtitles File 15.3 KB

Prim's
Algori
Spann

Impler
of Prin
Algori

Prim's Algorithm/Minimum Spanning Tree

Visit Course

Comment

K kartik + Follow 415

Article Tags : Graph Greedy DSA Amazon +4 More

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305